CITED REFERENCES AND FURTHER READING:

Numerical Recipes Software 2007, "Solvde Implementation," *Numerical Recipes Webnote No. 25*, at `http://www.nr.com/webnotes?25` [1]

Eggleton, P.P. 1971, "The Evolution of Low Mass Stars," *Monthly Notices of the Royal Astronomical Society*, vol. 151, pp. 351–364.[2]

Keller, H.B. 1968, *Numerical Methods for Two-Point Boundary-Value Problems*; reprinted 1991 (New York: Dover).

Kippenhan, R., Weigert, A., and Hofmeister, E. 1968, in *Methods in Computational Physics*, vol. 7 (New York: Academic Press), pp. 129ff.

18.4 A Worked Example: Spheroidal Harmonics

The best way to understand the algorithms of the previous sections is to see them employed to solve an actual problem. As a sample problem, we have selected the computation of spheroidal harmonics. (The more common name is spheroidal angle functions, but we prefer the explicit reminder of the kinship with spherical harmonics.) We will show how to find spheroidal harmonics, first by the method of relaxation (§18.3), and then by the methods of shooting (§18.1) and shooting to a fitting point (§18.2).

Spheroidal harmonics typically arise when certain partial differential equations are solved by separation of variables in spheroidal coordinates. They satisfy the following differential equation on the interval $-1 \le x \le 1$:

$$\frac{d}{dx}\left[(1-x^2)\frac{dS}{dx}\right] + \left(\lambda - c^2x^2 - \frac{m^2}{1-x^2}\right)S = 0 \tag{18.4.1}$$

Here m is an integer, c is the "oblateness parameter," and λ is the eigenvalue. Despite the notation, c^2 can be positive or negative. For $c^2 > 0$, the functions are called "prolate," while if $c^2 < 0$ they are called "oblate." The equation has singular points at $x = \pm 1$ and is to be solved subject to the boundary conditions that the solution be regular at $x = \pm 1$. Only for certain values of λ, the eigenvalues, will this be possible.

If we consider first the spherical case, where $c = 0$, we recognize the differential equation for Legendre functions $P_n^m(x)$. In this case the eigenvalues are $\lambda_{mn} = n(n+1)$, $n = m, m+1, \ldots$. The integer n labels successive eigenvalues for fixed m: When $n = m$ we have the lowest eigenvalue, and the corresponding eigenfunction has no nodes in the interval $-1 < x < 1$; when $n = m+1$ we have the next eigenvalue, and the eigenfunction has one node inside $(-1, 1)$; and so on.

A similar situation holds for the general case $c^2 \neq 0$. We write the eigenvalues of (18.4.1) as $\lambda_{mn}(c)$ and the eigenfunctions as $S_{mn}(x;c)$. For fixed m, $n = m, m+1, \ldots$ labels the successive eigenvalues.

The computation of $\lambda_{mn}(c)$ and $S_{mn}(x;c)$ traditionally has been quite difficult. Complicated recurrence relations, power series expansions, etc., can be found in [1-3]. Cheap computing makes evaluation by direct solution of the differential equation quite feasible.

The first step is to investigate the behavior of the solution near the singular points $x = \pm 1$. Substituting a power series expansion of the form

$$S = (1 \pm x)^{\alpha} \sum_{k=0}^{\infty} a_k (1 \pm x)^k \tag{18.4.2}$$

in equation (18.4.1), we find that the regular solution has $\alpha = m/2$. (Without loss of generality we can take $m \geq 0$ since $m \to -m$ is a symmetry of the equation.) We get an equation that is numerically more tractable if we factor out this behavior. Accordingly we set

$$S = (1 - x^2)^{m/2} y \tag{18.4.3}$$

We then find from (18.4.1) that y satisfies the equation

$$(1 - x^2)\frac{d^2 y}{dx^2} - 2(m+1)x\frac{dy}{dx} + (\mu - c^2 x^2)y = 0 \tag{18.4.4}$$

where

$$\mu \equiv \lambda - m(m+1) \tag{18.4.5}$$

Both equations (18.4.1) and (18.4.4) are invariant under the replacement $x \to -x$. Thus the functions S and y must also be invariant, except possibly for an overall scale factor. (Since the equations are linear, a constant multiple of a solution is also a solution.) Because the solutions will be normalized, the scale factor can only be ± 1. If $n - m$ is odd, there are an odd number of zeros in the interval $(-1, 1)$. Thus we must choose the antisymmetric solution $y(-x) = -y(x)$, which has a zero at $x = 0$. Conversely, if $n - m$ is even, we must have the symmetric solution. Thus

$$y_{mn}(-x) = (-1)^{n-m} y_{mn}(x) \tag{18.4.6}$$

and similarly for S_{mn}.

The boundary conditions on (18.4.4) require that y be regular at $x = \pm 1$. In other words, near the endpoints the solution takes the form

$$y = a_0 + a_1(1 - x^2) + a_2(1 - x^2)^2 + \cdots \tag{18.4.7}$$

Substituting this expansion in equation (18.4.4) and letting $x \to 1$, we find that

$$a_1 = -\frac{\mu - c^2}{4(m+1)} a_0 \tag{18.4.8}$$

Equivalently,

$$y'(1) = \frac{\mu - c^2}{2(m+1)} y(1) \tag{18.4.9}$$

A similar equation holds at $x = -1$ with a minus sign on the right-hand side. The irregular solution has a different relation between function and derivative at the endpoints.

Instead of integrating the equation from -1 to 1, we can exploit the symmetry (18.4.6) to integrate from 0 to 1. The boundary condition at $x = 0$ is

$$\begin{aligned} y(0) &= 0, \quad n - m \text{ odd} \\ y'(0) &= 0, \quad n - m \text{ even} \end{aligned} \tag{18.4.10}$$

A third boundary condition comes from the fact that any constant multiple of a solution y is a solution. We can thus *normalize* the solution. We adopt the normalization that the function S_{mn} has the same limiting behavior as P_n^m at $x = 1$:

$$\lim_{x \to 1} (1 - x^2)^{-m/2} S_{mn}(x; c) = \lim_{x \to 1} (1 - x^2)^{-m/2} P_n^m(x) \tag{18.4.11}$$

Various normalization conventions in the literature are tabulated by Flammer [1].

Imposing three boundary conditions for the second-order equation (18.4.4) turns it into an eigenvalue problem for λ or equivalently for μ. We write it in the standard form by setting

$$y_0 = y \tag{18.4.12}$$

$$y_1 = y' \tag{18.4.13}$$

$$y_2 = \mu \tag{18.4.14}$$

Then

$$y_0' = y_1 \tag{18.4.15}$$

$$y_1' = \frac{1}{1 - x^2} \left[2x(m+1)y_1 - (y_2 - c^2 x^2) y_0 \right] \tag{18.4.16}$$

$$y_2' = 0 \tag{18.4.17}$$

The boundary condition at $x = 0$ in this notation is

$$\begin{aligned} y_0 &= 0, \quad n - m \ \text{odd} \\ y_1 &= 0, \quad n - m \ \text{even} \end{aligned} \tag{18.4.18}$$

At $x = 1$ we have two conditions:

$$y_1 = \frac{y_2 - c^2}{2(m+1)} y_0 \tag{18.4.19}$$

$$y_0 = \lim_{x \to 1} (1 - x^2)^{-m/2} P_n^m(x) = \frac{(-1)^m (n+m)!}{2^m m! (n-m)!} \equiv \gamma \tag{18.4.20}$$

We are now ready to illustrate the use of the methods of previous sections on this problem.

18.4.1 Relaxation

If we just want a few isolated values of λ or S, shooting is probably the quickest method. However, if we want values for a large sequence of values of c, relaxation is better. Relaxation rewards a good initial guess with rapid convergence, and the previous solution should be a good initial guess if c is changed only slightly.

For simplicity, we choose a uniform grid on the interval $0 \le x \le 1$. For a total of M mesh points, we have

$$h = \frac{1}{M - 1} \tag{18.4.21}$$

$$x_k = kh, \qquad k = 0, 1, \ldots, M - 1 \tag{18.4.22}$$

At interior points $k = 1, 2, \ldots, M-1$, equation (18.4.15) gives

$$E_{0,k} = y_{0,k} - y_{0,k-1} - \frac{h}{2}(y_{1,k} + y_{1,k-1}) \tag{18.4.23}$$

Equation (18.4.16) gives

$$\begin{aligned} E_{1,k} &= y_{1,k} - y_{1,k-1} - \beta_k \\ &\times \left[\frac{(x_k + x_{k-1})(m+1)(y_{1,k} + y_{1,k-1})}{2} - \alpha_k \frac{(y_{0,k} + y_{0,k-1})}{2} \right] \end{aligned} \tag{18.4.24}$$

where

$$\alpha_k = \frac{y_{2,k} + y_{2,k-1}}{2} - \frac{c^2(x_k + x_{k-1})^2}{4} \tag{18.4.25}$$

$$\beta_k = \frac{h}{1 - \frac{1}{4}(x_k + x_{k-1})^2} \tag{18.4.26}$$

Finally, equation (18.4.17) gives

$$E_{2,k} = y_{2,k} - y_{2,k-1} \tag{18.4.27}$$

Now recall that the matrix of partial derivatives $S_{i,j}$ of equation (18.3.8) is defined so that i labels the equation and j the variable. In our case, j runs from 0 to 2 for y_j at $k-1$ and from 3 to 5 for y_j at k. Thus equation (18.4.23) gives

$$\begin{aligned} S_{0,0} &= -1, \qquad S_{0,1} = -\frac{h}{2}, \qquad S_{0,2} = 0 \\ S_{0,3} &= 1, \qquad S_{0,4} = -\frac{h}{2}, \qquad S_{0,5} = 0 \end{aligned} \tag{18.4.28}$$

Similarly equation (18.4.24) yields

$$\begin{aligned} S_{1,0} &= \alpha_k \beta_k / 2, & S_{1,1} &= -1 - \beta_k (x_k + x_{k-1})(m+1)/2, \\ S_{1,2} &= \beta_k (y_{0,k} + y_{0,k-1})/4, & S_{1,3} &= S_{1,0}, \\ S_{1,4} &= 2 + S_{1,1}, & S_{1,5} &= S_{1,2} \end{aligned} \tag{18.4.29}$$

while from equation (18.4.27) we find

$$\begin{aligned} S_{2,0} &= 0, \qquad S_{2,1} = 0, \qquad S_{2,2} = -1 \\ S_{2,3} &= 0, \qquad S_{2,4} = 0, \qquad S_{2,5} = 1 \end{aligned} \tag{18.4.30}$$

At $x = 0$ we have the boundary condition

$$E_{2,0} = \begin{cases} y_{0,0}, & n-m \text{ odd} \\ y_{1,0}, & n-m \text{ even} \end{cases} \tag{18.4.31}$$

Recall the convention adopted in the `solvde` routine that for one boundary condition at $k = 0$ only $S_{2,j}$ can be nonzero. Also, j takes on the values 3 to 5 since the boundary condition involves only y_k, not y_{k-1}. Accordingly, the only nonzero values of $S_{2,j}$ at $x = 0$ are

$$\begin{aligned} S_{2,3} &= 1, \qquad n-m \text{ odd} \\ S_{2,4} &= 1, \qquad n-m \text{ even} \end{aligned} \tag{18.4.32}$$

At $x = 1$ we have

$$E_{0,M} = y_{1,M-1} - \frac{y_{2,M-1} - c^2}{2(m+1)} y_{0,M-1} \tag{18.4.33}$$

$$E_{1,M} = y_{0,M-1} - \gamma \tag{18.4.34}$$

Thus

$$S_{0,3} = -\frac{y_{2,M-1} - c^2}{2(m+1)}, \qquad S_{0,4} = 1, \qquad S_{0,5} = -\frac{y_{0,M-1}}{2(m+1)} \tag{18.4.35}$$

$$S_{1,3} = 1, \qquad S_{1,4} = 0, \qquad S_{1,5} = 0 \tag{18.4.36}$$

Here now is the sample program that implements the above algorithm. We need a main program, sfroid, that calls the routine Solvde, and we must supply the object Difeq to be passed to Solvde. For simplicity we choose an equally spaced mesh of m = 41 points, that is, $h = .025$. As we shall see, this gives good accuracy for the eigenvalues up to moderate values of $n - m$.

Since the boundary condition at $x = 0$ does not involve y_0 if $n - m$ is even, we have to use the indexv feature of Solvde. Recall that the value of indexv[j] describes which column of s[i][j] the variable y[j] has been put in. If $n - m$ is even, we need to interchange the columns for y_0 and y_1 so that there is not a zero pivot element in s[i][j].

The program prompts for values of m and n. It then computes an initial guess for y based on the Legendre function P_n^m. It next prompts for c^2, solves for y, prompts for c^2, solves for y using the previous values as an initial guess, and so on.

sfroid.h

```
Int main_sfroid(void)
```
Sample program using Solvde. Computes eigenvalues of spheroidal harmonics $S_{mn}(x;c)$ for $m \geq 0$ and $n \geq m$. In the program, m is mm, c^2 is c2, and γ of equation (18.4.20) is anorm.

```
{
    const Int M=40,MM=4;
    const Int NE=3,NB=1,NYJ=NE,NYK=M+1;
    Int mm=3,n=5,mpt=M+1;
    VecInt indexv(NE);
    VecDoub x(M+1),scalv(NE);
    MatDoub y(NYJ,NYK);
    Int itmax=100;
    Doub c2[]={16.0,20.0,-16.0,-20.0};
    Doub conv=1.0e-14,slowc=1.0,h=1.0/M;
    if ((n+mm & 1) != 0) {                          No interchanges necessary.
        indexv[0]=0;
        indexv[1]=1;
        indexv[2]=2;
    } else {                                        Interchange y0 and y1.
        indexv[0]=1;
        indexv[1]=0;
        indexv[2]=2;
    }
    Doub anorm=1.0;                                 Compute γ.
    if (mm != 0) {
        Doub q1=n;
        for (Int i=1;i<=mm;i++) anorm = -0.5*anorm*(n+i)*(q1--/i);
    }
    for (Int k=0;k<M;k++) {                         Initial guess.
        x[k]=k*h;
        Doub fac1=1.0-x[k]*x[k];
        Doub fac2=exp((-mm/2.0)*log(fac1));
```

```
        y[0][k]=plgndr(n,mm,x[k])*fac2;                   P_n^m from §6.7.
        Doub deriv = -((n-mm+1)*plgndr(n+1,mm,x[k])-      Derivative of P_n^m from a recur-
            (n+1)*x[k]*plgndr(n,mm,x[k]))/fac1;              rence relation.
        y[1][k]=mm*x[k]*y[0][k]/fac1+deriv*fac2;
        y[2][k]=n*(n+1)-mm*(mm+1);
    }
    x[M]=1.0;                                             Initial guess at x = 1 done sep-
    y[0][M]=anorm;                                           arately.
    y[2][M]=n*(n+1)-mm*(mm+1);
    y[1][M]=y[2][M]*y[0][M]/(2.0*(mm+1.0));
    scalv[0]=abs(anorm);                                  Set scaling.
    scalv[1]=(y[1][M] > scalv[0] ? y[1][M] : scalv[0]);
    scalv[2]=(y[2][M] > 1.0 ? y[2][M] : 1.0);
    for (Int j=0;j<MM;j++) {
        Difeq difeq(mm,n,mpt,h,c2[j],anorm,x);            Set up Difeq object.
        Solvde solvde(itmax,conv,slowc,scalv,indexv,NB,y,difeq);
        cout << endl << " m = " << setw(3) << mm;
        cout << "  n = " << setw(3) << n << "  c**2 = ";
        cout << fixed << setprecision(3) << setw(7) << c2[j];
        cout << " lamda = " << setprecision(6) << (y[2][0]+mm*(mm+1));
        cout << endl;                                     Return for another value of c^2.
    }
    return 0;
}
```

difeq.h

```
struct Difeq {
```

Provides matrix s for Solvde.

```
    const Int &mm,&n,&mpt;                          These variables are defined in sfroid.
    const Doub &h,&c2,&anorm;
    const VecDoub &x;
    Difeq(const Int &mmm, const Int &nn, const Int &mptt, const Doub &hh,
        const Doub &cc2, const Doub &anormm, VecDoub_I &xx) : mm(mmm),
        n(nn), mpt(mptt), h(hh), c2(cc2), anorm(anormm), x(xx) {}

    void smatrix(const Int k, const Int k1, const Int k2, const Int jsf,
        const Int is1, const Int isf, VecInt_I &indexv, MatDoub_O &s,
        MatDoub_I &y)
```

Returns matrix s for solvde.

```
    {
        Doub temp,temp1,temp2;

        if (k == k1) {                              Boundary condition at first point.
            if ((n+mm & 1) != 0) {
                s[2][3+indexv[0]]=1.0;              Equation (18.4.32).
                s[2][3+indexv[1]]=0.0;
                s[2][3+indexv[2]]=0.0;
                s[2][jsf]=y[0][0];                  Equation (18.4.31).
            } else {
                s[2][3+indexv[0]]=0.0;              Equation (18.4.32).
                s[2][3+indexv[1]]=1.0;
                s[2][3+indexv[2]]=0.0;
                s[2][jsf]=y[1][0];                  Equation (18.4.31).
            }
        } else if (k > k2-1) {                      Boundary conditions at last point.
            s[0][3+indexv[0]] = -(y[2][mpt-1]-c2)/(2.0*(mm+1.0));      (18.4.35).
            s[0][3+indexv[1]]=1.0;
            s[0][3+indexv[2]] = -y[0][mpt-1]/(2.0*(mm+1.0));
            s[0][jsf]=y[1][mpt-1]-(y[2][mpt-1]-c2)*y[0][mpt-1]/        (18.4.33).
                (2.0*(mm+1.0));
            s[1][3+indexv[0]]=1.0;                  Equation (18.4.36).
            s[1][3+indexv[1]]=0.0;
            s[1][3+indexv[2]]=0.0;
```

```
            s[1][jsf]=y[0][mpt-1]-anorm;             Equation (18.4.34).
        } else {                                     Interior point.
            s[0][indexv[0]] = -1.0;                  Equation (18.4.28).
            s[0][indexv[1]] = -0.5*h;
            s[0][indexv[2]]=0.0;
            s[0][3+indexv[0]]=1.0;
            s[0][3+indexv[1]] = -0.5*h;
            s[0][3+indexv[2]]=0.0;
            temp1=x[k]+x[k-1];
            temp=h/(1.0-temp1*temp1*0.25);
            temp2=0.5*(y[2][k]+y[2][k-1])-c2*0.25*temp1*temp1;
            s[1][indexv[0]]=temp*temp2*0.5;          Equation (18.4.29).
            s[1][indexv[1]] = -1.0-0.5*temp*(mm+1.0)*temp1;
            s[1][indexv[2]]=0.25*temp*(y[0][k]+y[0][k-1]);
            s[1][3+indexv[0]]=s[1][indexv[0]];
            s[1][3+indexv[1]]=2.0+s[1][indexv[1]];
            s[1][3+indexv[2]]=s[1][indexv[2]];
            s[2][indexv[0]]=0.0;                     Equation (18.4.30).
            s[2][indexv[1]]=0.0;
            s[2][indexv[2]] = -1.0;
            s[2][3+indexv[0]]=0.0;
            s[2][3+indexv[1]]=0.0;
            s[2][3+indexv[2]]=1.0;
            s[0][jsf]=y[0][k]-y[0][k-1]-0.5*h*(y[1][k]+y[1][k-1]);      (18.4.23).
            s[1][jsf]=y[1][k]-y[1][k-1]-temp*((x[k]+x[k-1])            (18.4.24).
                *0.5*(mm+1.0)*(y[1][k]+y[1][k-1])-temp2
                *0.5*(y[0][k]+y[0][k-1]));
            s[2][jsf]=y[2][k]-y[2][k-1];             Equation (18.4.27).
        }
    }
};
```

You can run the program and check it against values of $\lambda_{mn}(c)$ given in the tables at the back of Flammer's book [1] or in Table 21.1 of Abramowitz and Stegun [2]. Typically it converges in about three iterations. The table below gives a few comparisons.

Selected Output of `sfroid`				
m	n	c^2	λ_{exact}	$\lambda_{\texttt{sfroid}}$
2	2	0.1	6.01427	6.01427
		1.0	6.14095	6.14095
		4.0	6.54250	6.54253
2	5	1.0	30.4361	30.4372
		16.0	36.9963	37.0135
4	11	-1.0	131.560	131.554

18.4.2 Shooting

To solve the same problem via shooting (§18.1), we supply a functor `Rhs` that implements equations (18.4.15) – (18.4.17). We will integrate the equations over the range $-1 \le x \le 0$. We provide the functor `Load`, which sets the eigenvalue y_2 to its current best estimate, `v[0]`. It also sets the boundary values of y_0 and y_1 using equations (18.4.20) and (18.4.19) (with a minus sign corresponding to $x = -1$). Note that the boundary condition is actually applied a distance `dx` from the boundary to avoid having to evaluate y_1' right on the boundary. The functor Score follows from equation (18.4.18).

sphoot.h

```
struct Rhs {
Evaluates derivatives for Odeint.
    Int m;
    Doub c2;
    Rhs(Int mm, Doub cc2) : m(mm), c2(cc2) {}
    Constructor gets parameters from main.
    void operator() (const Doub x, VecDoub_I &y, VecDoub_O &dydx)
    {
        dydx[0]=y[1];
        dydx[1]=(2.0*x*(m+1.0)*y[1]-(y[2]-c2*x*x)*y[0])/(1.0-x*x);
        dydx[2]=0.0;
    }
};

struct Load {
Supplies starting values for integration at x = -1 + dx.
    Int n,m;
    Doub gmma,c2,dx;
    VecDoub y;
    Load(Int nn, Int mm, Doub gmmaa, Doub cc2, Doub dxx) : n(nn), m(mm),
        gmma(gmmaa), c2(cc2), dx(dxx), y(3) {}
    Constructor gets parameters from main.
    VecDoub operator() (const Doub x1, VecDoub_I &v)
    {
        Doub y1 = ((n-m & 1) != 0 ? -gmma : gmma);
        y[2]=v[0];
        y[1] = -(y[2]-c2)*y1/(2*(m+1));
        y[0]=y1+y[1]*dx;
        return y;
    }
};

struct Score {
Computes amount by which boundary condition at x = 0 is violated.
    Int n,m;
    VecDoub f;
    Score(Int nn, Int mm) : n(nn), m(mm), f(1) {}
    Constructor gets parameters from main.
    VecDoub operator() (const Doub xf, VecDoub_I &y)
    {
        f[0]=((n-m & 1) != 0 ? y[0] : y[1]);
        return f;
    }
};

Int main_sphoot(void) {
Sample program using Shoot. Computes eigenvalues of spheroidal harmonics S_mn(x;c) for
m >= 0 and n >= m. Note how the functor vecfunc for newt is provided by Shoot (§18.1).
    const Int N2=1,MM=3;
    Bool check;
    VecDoub v(N2);
    Int j,m=3,n=5;
    Doub c2[]={1.5,-1.5,0.0};
    Int nvar=3;                         Number of equations.
    Doub dx=1.0e-8;                     Avoid evaluating derivatives exactly at x = -1.
    for (j=0;j<MM;j++) {
        Doub gmma=1.0;                      Compute γ of equation (18.4.20).
        Doub q1=n;
        for (Int i=1;i<=m;i++) gmma *= -0.5*(n+i)*(q1--/i);
        v[0]=n*(n+1)-m*(m+1)+c2[j]/2.0;     Initial guess for eigenvalue.
```

```
        Doub x1= -1.0+dx;                    Set range of integration.
        Doub x2=0.0;
        Load load(n,m,gmma,c2[j],dx);        Set up Load, Rhs, and Score objects ...
        Rhs d(m,c2[j]);
        Score score(n,m);                    ... use them to set up Shoot object ...
        Shoot<Load,Rhs,Score> shoot(nvar,x1,x2,load,d,score);
        newt(v,check,shoot);                 ... and use it to find v that zeros vector f in
        if (check) {                            Score.
            cout << "shoot failed; bad initial guess" << endl;
        } else {
            cout << "    " << "mu(m,n)" << endl;
            cout << fixed << setprecision(6);
            cout << setw(12) << v[0] << endl;
        }
    }
    return 0;
}
```

18.4.3 Shooting to a Fitting Point

For variety we illustrate Shootf from §18.2 by integrating over the whole range $-1 + dx \le x \le 1 - dx$, with the fitting point chosen to be at $x = 0$. The routine Rhsfpt is identical to Rhs for Shoot since we are integrating the same equation. Now, however, there are two load routines. The functor Load1 for $x = -1$ is essentially identical to Load above. At $x = 1$, Load2 sets the function value y_0 and the eigenvalue y_2 to their best current estimates, v2[0] and v2[1], respectively. If you quite sensibly make your initial guess of the eigenvalue the same in the two intervals, then v1[0] will stay equal to v2[1] during the iteration. The functor Score computes the degree of mismatch of the three function values at the fitting point.

sphfpt.h

```
struct Rhsfpt {
    Int m;
    Doub c2;
    Rhsfpt(Int mm, Doub cc2) : m(mm), c2(cc2) {}
    void operator() (const Doub x, VecDoub_I &y, VecDoub_O &dydx)
    {
        dydx[0]=y[1];
        dydx[1]=(2.0*x*(m+1.0)*y[1]-(y[2]-c2*x*x)*y[0])/(1.0-x*x);
        dydx[2]=0.0;
    }
};

struct Load1 {
Supplies starting values for integration at x = -1 + dx.
    Int n,m;
    Doub gmma,c2,dx;
    VecDoub y;
    Load1(Int nn, Int mm, Doub gmmaa, Doub cc2, Doub dxx) : n(nn), m(mm),
        gmma(gmmaa), c2(cc2), dx(dxx), y(3) {}
    VecDoub operator() (const Doub x1, VecDoub_I &v1)
    {
        Doub y1 = ((n-m & 1) != 0 ? -gmma : gmma);
        y[2]=v1[0];
        y[1] = -(y[2]-c2)*y1/(2*(m+1));
        y[0]=y1+y[1]*dx;
        return y;
    }
};
```

```
struct Load2 {
```
Supplies starting values for integration at $x = 1 - dx$.
```
    Int m;
    Doub c2;
    VecDoub y;
    Load2(Int mm, Doub cc2) : m(mm), c2(cc2), y(3) {}
    VecDoub operator() (const Doub x2, VecDoub_I &v2)
    {
        y[2]=v2[1];
        y[0]=v2[0];
        y[1]=(y[2]-c2)*y[0]/(2*(m+1));
        return y;
    }
};
```

```
struct Score {
```
Computes the mismatch of the solutions at the fitting point $x = 0$.
```
    VecDoub f;
    Score() : f(3) {}
    VecDoub operator() (const Doub xf, VecDoub_I &y)
    {
        for (Int i=0;i<3;i++) f[i]=y[i];
        return f;
    }
};
```

```
Int main_sphfpt(void) {
```
Sample program using `Shootf`. Computes eigenvalues of spheroidal harmonics $S_{mn}(x;c)$ for $m \geq 0$ and $n \geq m$. Note how the functor `vecfunc` for `newt` is provided by `Shootf` (§18.2). The routine `Rhsfpt` is the same as `Rhs` for `sphoot`.
```
    const Int N1=2,N2=1,NTOT=N1+N2,MM=3;
    Bool check;
    VecDoub v(NTOT);
    Int j,m=3,n=5,n2=N2;
    Doub c2[]={1.5,-1.5,0.0};
    Int nvar=NTOT;                          Number of equations.
    Doub dx=1.0e-8;                         Avoid evaluating derivatives exactly at x = ±1.
    for (j=0;j<MM;j++) {
        Doub gmma=1.0;                      Compute γ of equation (18.4.20).
        Doub q1=n;
        for (Int i=1;i<=m;i++) gmma *= -0.5*(n+i)*(q1--/i);
        v[0]=n*(n+1)-m*(m+1)+c2[j]/2.0;Initial guess for eigenvalue and function value.
        v[2]=v[0];
        v[1]=gmma*(1.0-(v[2]-c2[j])*dx/(2*(m+1)));
        Doub x1= -1.0+dx;                   Set range of integration.
        Doub x2=1.0-dx;
        Doub xf=0.0;                        Fitting point.
        Load1 load1(n,m,gmma,c2[j],dx);Set up Load1, Load2, Rhsfpt, and Score
        Load2 load2(m,c2[j]);                   objects ...
        Rhsfpt d(m,c2[j]);
        Score score;
        Shootf<Load1,Load2,Rhsfpt,Score> shootf(nvar,n2,x1,x2,xf,load1,
            load2,d,score);                 ... use them to set up Shootf object ...
        newt(v,check,shootf);               ... and use it to find v that zeros vector f in
        if (check) {                            Score.
            cout << "shootf failed; bad initial guess" << endl;
        } else {
            cout << "    " << "mu(m,n)" << endl;
            cout << fixed << setprecision(6);
            cout << setw(12) << v[0] << endl;
        }
    }
    return 0;
}
```